# Overview

This was the first session of an AI Engineering Bootcamp led by Rahul Tiwari, Lead AI Engineer at EY, with over 8 years of experience in AI and ML. The session focused on introducing participants to the bootcamp structure, setting up the development environment, and providing foundational context for building AI agents. Rahul outlined his career journey from Java developer to AI/ML engineer and emphasized a hands-on, industry-aligned approach throughout the course.

## Key Concepts or Theories

- **LLMs (Large Language Models):** AI models accessed via API or SDK from providers such as OpenAI, Anthropic, and Google Gemini, functioning as stateless prediction engines without built-in memory
- **AI Agents:** Autonomous systems composed of a brain (LLM), tools (actions), instructions (prompts), and memory, capable of making decisions and performing tasks
- **Virtual Environments:** Isolated Python environments per project to manage library versions and avoid dependency conflicts
- **API Keys:** Authentication credentials required to access LLM providers' models programmatically
- **.env Files:** Secure storage for sensitive credentials, excluded from version control via .gitignore
- **Python SDK vs. API Calling:** Two methods to access LLM models; Python SDK offers a simpler, ready-to-use interface

## Important Questions Raised

- Do we really need AI agents for a given problem, or can a simpler Python function suffice?
- How do we evaluate the return on investment (ROI) when building AI agent solutions?
- How does model versioning and library deprecation affect long-term code stability?
- How can we ensure security and compliance when integrating LLMs into production systems?
- How should agents handle looping behavior or unresolvable situations?

---

# Key Takeaways and Summary of Learning Objectives

- The bootcamp is entirely hands-on, following industry best practices used by professional AI engineers
- Understanding when to use AI agents versus simpler solutions is a critical and often overlooked skill
- AI agents have existed conceptually for 70 years; the course will trace their evolution from rule engines to LLM-based agents
- Participants will build a Code Review Agent as the primary project, using pure Python without predefined frameworks initially
- Development environment setup is foundational: Visual Studio Code, Python 3.9+, Git Bash, and a GitHub account are required
- Virtual environments must be used per project to maintain library version consistency

- API keys must never be hard-coded in source files; they should be stored in .env files and excluded from Git
- LLM providers (OpenAI, Anthropic, Gemini) each have different SDKs and calling conventions; LangChain addresses this inconsistency
- Security, compliance, and ROI considerations will be integrated throughout the course as projects mature

---

# Topic 1: Development Environment Setup

Participants were guided through setting up the core tools required for the bootcamp. The three essential tools identified were Visual Studio Code (code editor), Python 3.9 or higher (runtime), and Git Bash (cross-platform terminal for Git operations). Rahul emphasized using Git Bash specifically to ensure command compatibility across Windows and macOS systems, as native terminals such as PowerShell may not support all required commands.
A GitHub account is also required, as all course code and notes will be maintained in a shared repository. Rahul demonstrated how to clone or download the repository and explained that a GitHub repository functions as an online folder for collaborative code management.
Participants were shown how to create a local project folder, open it in Visual Studio Code, and navigate the integrated terminal. The shared course repository is titled AI for Nokri, and all session code will be organized within it by session number.

### Relevant Q&A

Question: "Do we need to download Homebrew for Git Bash on Mac?"
Answer: Yes, it is mandatory for Mac users. Follow the official documentation to install it.
Question: "Can we use PyCharm instead of Visual Studio Code?"
Answer: Yes, PyCharm is also acceptable as a development environment.
Question: "What is the command to activate the virtual environment on Windows versus Mac?"
Answer: For Mac, use source AI-ENV/bin/activate. For Windows (in Git Bash), use source AI-ENV/scripts/activate. The virtual environment is created using the command python -m venv AI-ENV on both systems.

---

# Topic 2: Accessing LLM Models via OpenAI SDK

Rahul explained the distinction between AI applications (such as ChatGPT) and the underlying LLM models. ChatGPT is a fully built application with memory, storage, and a user interface, whereas the model itself is a stateless component that simply takes input and returns a prediction. For enterprise use cases, developers should access the model directly via API or SDK rather than using third-party applications, to maintain data compliance and control.
Participants were shown how to create an OpenAI API key at platform.openai.com, set it to a 30-day validity period, and store it securely in a .env file. The python-dotenv library is used to load this key at runtime without exposing it in source code. Rahul demonstrated a working llms.py file that imports the OpenAI library, loads credentials from the .env file, and calls the GPT-4o Mini model with a simple prompt, confirming a successful API connection.
The importance of pinning library versions was highlighted: if the OpenAI Python library is updated and a method is renamed or removed, existing code will break. Virtual

environments and a requirements.txt file (listing openai and python-dotenv) are used to lock and reproduce the exact dependency state across environments.

### Relevant Q&A

Question: "Why do we keep credentials in a .env file?"
Answer: Mubin Shaikh noted that .env files are excluded from Git synchronization. Arnab Halder added that environment variables can be loaded programmatically at runtime, keeping secrets out of source code.
Question: "How do we know which model names are available and valid?"
Answer: Always refer to the official OpenAI platform documentation at platform.openai.com, where a full list of available and deprecated models is maintained. Older models such as GPT-4 (base) have been deprecated and replaced by newer variants such as GPT-4o Mini.
Question: "Does the code structure for calling a model change between providers such as OpenAI and Anthropic?"
Answer: Yes. Each provider has its own SDK and calling conventions. The message structure (role and content fields) remains consistent, but the client initialization and method names differ. LangChain solves this by providing a unified interface that abstracts provider-specific differences.

---

## Topic 3: Problem Statement — Code Review Agent

Rahul introduced the primary project for the bootcamp: a Code Review Agent. In a typical software development team, lead developers manually review pull requests to check for coding standard compliance, security vulnerabilities, and potential bugs before merging code into the main branch. This process is time-consuming and scales poorly as team size grows.
The proposed agent automates this workflow. When a developer raises a pull request, the agent is triggered, reads the code changes, compares them against predefined team coding standards and security rules, and either approves the merge or blocks it with detailed comments explaining the issues found. The agent uses an LLM as its reasoning brain, a set of tools to perform specific checks, predefined instructions (system prompts), and optionally memory to track recurring issues from the same developer.
Rahul demonstrated a working prototype of this agent on his own GitHub repository, showing how it posts inline comments on pull requests and can block merging when issues are detected. The full implementation will be built during the Saturday session using OpenAI models and pure Python, without relying on high-level frameworks initially.

### Relevant Q&A

Question: "Will the agent compare one developer's code against another developer's code?"
Answer: No. The agent compares each developer's code against the team's predefined coding standards and security rules, not against other developers' work.
Question: "Will the agent also suggest mitigation steps for security vulnerabilities it finds?"
Answer: The current scope of the agent is limited to review and flagging. Mitigation suggestions would be handled by a separate, additional agent built on top of the review agent.
Question: "How do we handle situations where the agent gets caught in a loop?"
Answer: A maximum retry limit (max_try) can be configured. If the agent exceeds this limit, the pull request is escalated to a human developer for manual review. Full human oversight is always maintained.
Question: "Is this similar to GitHub Copilot?"

Answer: Conceptually similar, but the key difference is ownership and customization. Organizations in regulated industries such as healthcare may not be permitted to use GitHub Copilot. This bootcamp teaches participants how to build an equivalent solution independently, tailored to their organization's specific standards.

---

# Actionable Next Steps / Assignments

- Install Visual Studio Code, Python 3.9+, and Git Bash before the next session
- Create a GitHub account if not already done
- Clone or download the AI for Nokri repository shared in the session chat
- Create a local project folder, set up a virtual environment using python -m venv AI-ENV, and activate it
- Create a requirements.txt file containing openai and python-dotenv, then install dependencies using pip install -r requirements.txt
- Create a .env file and store the shared OpenAI API key in it; verify the setup by running the llms.py file and confirming a successful model response
- Participants who were unable to complete the setup should join the Saturday session at 4:30 PM (30 minutes early) for dedicated setup assistance
- All participants should have a working development environment ready before the Saturday session, where the first AI agent will be built

---

# Supplemental Resources

- GitHub Repository — AI for Nokri: The shared course repository containing all session code, notes, and README files with step-by-step setup commands. URL shared in the session chat.
- Visual Studio Code Download: https://code.visualstudio.com — Select the appropriate installer for your operating system (Windows, macOS, or Linux)
- Python Official Download Page: https://python.org — Download the latest stable version (3.9 or higher recommended)
- Git Bash Download: https://git-scm.com — For Windows users, select the Git for Windows x64 Setup installer
- OpenAI Platform: https://platform.openai.com — Used to create and manage API keys and view the full list of available models
- PyPI (Python Package Index): https://pypi.org — Official source for verifying correct library names and versions before adding them to requirements.txt
- OpenAI Python SDK Documentation: Available on the OpenAI platform; includes ready-to-use code samples for model calls that can be adapted directly into project files